

Nome: Guilherme de Moraes China Costa

Data: 26/09/2026

Título: Projeto de Bloco: Engenharia de Softwares Escaláveis — TP4

Refatoração para Arquitetura Orientada a Eventos

Spring Boot · RabbitMQ · Transactional Outbox · Retry e DLQ

Link da implementação:

https://github.com/GuiC0506/infnet/tree/master/projeto_bloco_engenharia_software_escalavel/TP4

2

eventos de domínio
versionados

4

padrões de confiabilidade

9/9

testes automatizados
aprovados

Resultado: a criação de empréstimos e devoluções não depende mais da disponibilidade imediata do microserviço de notificações.

1. Mudança arquitetural

Antes: comunicação fortemente acoplada

O backend chamava o endpoint POST de Notificações via OpenFeign dentro da operação de empréstimo. Falha ou lentidão no microserviço propagava-se para a transação principal.



Depois: escrita orientada a eventos



A consulta GET continua síncrona, pois é uma leitura solicitada pelo usuário. A escrita foi desacoplada e passa pelo broker.

Componentes do Docker Compose

Componente	Responsabilidade	Porta
React	Interface da biblioteca	4273
Biblioteca	Catálogo, leitores, circulação e publicação	8280
RabbitMQ	Roteamento, fila durável, ACK e DLQ	5672 / 15672
Notificações	Consumo idempotente e histórico	8281

2. Fluxo dos eventos

- 1 A API valida livro e leitor e atualiza o empréstimo.
- 2 O evento é serializado e salvo em `event_outbox` na mesma transação do domínio.
- 3 `OutboxPublisher`, executado por `@Scheduled`, envia JSON persistente ao `TopicExchange`.
- 4 O publisher só marca o evento como PUBLICADO após receber o publisher confirm do RabbitMQ.
- 5 A routing key seleciona a fila `notificacoes.emprestimos.v1`.
- 6 O método com `@RabbitListener` consome e verifica o `eventId`.
- 7 A notificação é persistida e a mensagem recebe ACK. Em falha, há três tentativas e depois DLQ.

Eventos e roteamento

Evento	Routing key	Efeito
EMPRESTIMO_REGISTRADO	biblioteca.emprestimo.registrado.v1	Aviso com data prevista.
DEVOLUCAO_REGISTRADA	biblioteca.emprestimo.devolido.v1	Confirmação de devolução.

```
{
  "eventId": "0e62...c7",
  "eventType": "EMPRESTIMO_REGISTRADO",
  "eventVersion": 1,
  "occurredAt": "2026-09-26T16:55:26Z",
  "emprestimoId": 1,
  "leitorId": 3,
  "livroTitulo": "Arquitetura Orientada a Eventos",
  "dataPrevistaDevolucao": "2026-10-30"
}
```

3. Padrões de mensagens e Spring Boot

Padrão	Implementação	Benefício
Publish/subscribe	TopicExchange + routing keys	Novos consumidores sem alterar o produtor.
Fila de trabalho	Fila durável com consumidores concorrentes	Distribuição de carga e escala horizontal.
Transactional Outbox	Evento e domínio no mesmo H2/commit	Não perde eventos quando o broker falha.
Retry + Dead Letter	3 tentativas, backoff, DLX e DLQ	Isola mensagens problemáticas.
Idempotent Consumer	eventId com unicidade	Reentregas não duplicam avisos.
Request/response	OpenFeign somente para GET	Leitura imediata para o frontend.

Abstrações utilizadas

- `spring-boot-starter-amqp`
- `RabbitTemplate`
- `@RabbitListener`
- `TopicExchange`
- `Queue` e `BindingBuilder`
- `@Scheduled`
- `publisher confirms`
- configuração externa por ambiente

Filas e exchanges são declaradas automaticamente na inicialização. As mensagens são persistentes e os eventos possuem contrato versionado.

4. Avaliação: prós, contras e cenários

Vantagens

- Desacoplamento temporal entre produtor e consumidor.
- Escala horizontal e absorção de picos.
- Extensibilidade para novos consumidores.
- Falhas recuperáveis com Outbox, retry e DLQ.
- Responsabilidades e bancos independentes.

Desvantagens

- Consistência eventual.
- Mais infraestrutura para operar.
- Depuração distribuída mais difícil.
- Necessidade de versionar contratos.
- Entrega pelo menos uma vez exige idempotência.

Quando é vantajosa

Integrações assíncronas, múltiplos interessados no mesmo fato, picos de carga, processos demorados e domínios em que o produtor deve continuar disponível mesmo se consumidores falharem.

Quando evitar

Operações pequenas que precisam de resposta imediata e consistência forte entre poucos componentes podem ser mais simples com comunicação síncrona.

Mitigações adotadas

Risco	Mitigação
Perda entre banco e broker	Transactional Outbox.
Duplicação	eventId único e consumidor idempotente.
Mensagem inválida	Retry com backoff e dead-letter queue.
Contrato incompatível	eventVersion e routing key terminada em v1.

5. Demonstração funcional

Sistema de Biblioteca

Catálogo, leitores e circulação em uma interface simples para o dia a dia da biblioteca.

0 livros disponíveis agora

0 empréstimos ativos

Failed to fetch

Catálogo Leitores Circulação

Catálogo

Cadastre livros, acompanhe a disponibilidade e abra o histórico de cada exemplar.

Título

Autor

ISBN

Cadastrar livro

Interface React executada pelo Docker Compose durante a validação.

✓ Cenário 1 — caminho feliz

Empréstimo do livro “Arquitetura Orientada a Eventos” gerou EMPRESTIMO_REGISTRADO. A devolução gerou DEVOLUCAO_REGISTRADA. As duas notificações ficaram disponíveis na consulta do leitor.

✓ Cenário 2 — broker indisponível

Com RabbitMQ parado, o empréstimo “Sistemas Resilientes” retornou sucesso e a consulta inicialmente retornou lista vazia. Após reiniciar o broker, a notificação apareceu automaticamente, comprovando a recuperação pela Outbox.

6. Testes e evidências

6/6

backend

3/3

notificações

0

falhas

```
backend
Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS

notificacoes-service
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS

docker compose config
Configuração válida
```

O que os testes verificam

- registro do evento de empréstimo na Outbox;
- routing key e conteúdo do evento;
- persistência da notificação recebida por evento;
- idempotência diante do mesmo eventId;
- repositórios, regras de circulação e histórico já existentes.

Comandos de reprodução

```
docker compose up --build
cd backend && mvn test
cd ../notificacoes-service && mvn test
```

O arquivo `ROTEIRO_DEMONSTRACAO.md` contém a sequência pronta para a gravação, incluindo o cenário com o RabbitMQ temporariamente indisponível.

7. Atendimento à rubrica

Item	Evidência entregue
Prós e contras	Seção 4 deste relatório e README.
Diferentes padrões	Pub/sub, work queue, Outbox, retry/DLQ, idempotência e request/response.
RabbitMQ	Exchange, bindings, filas duráveis, confirm, ACK, retry e DLQ.
Abstrações Spring Boot	RabbitTemplate, @RabbitListener, @Scheduled e Beans AMQP.
Sistema refatorado	POST síncrono removido; produtor e consumidor desacoplados.
Código-fonte	Backend, frontend, microserviço e Compose no TP4.
Git	Arquivos versionados no repositório Git.
Documentação e diagramas	README, arquitetura, sequência, contrato e este PDF.
Apresentação prática	Roteiro de 5–7 minutos e cenários previamente validados.

Conclusão

A refatoração eliminou o acoplamento síncrono na escrita e aumentou resiliência, escalabilidade e capacidade de evolução. A solução lida explicitamente com os principais desafios de sistemas orientados a eventos: perda, duplicação, falha persistente e evolução de contrato.

Material para gravação: execute o roteiro, mostre a interface e o painel RabbitMQ e finalize com o cenário de indisponibilidade. A gravação em si fica a cargo do aluno, conforme combinado.